

CHƯƠNG 4. TÍNH ĐA HÌNH (TÍNH TƯƠNG ỨNG BỘI)

1

TS. Phạm Minh Hoàn
Viện Công nghệ thông tin Kinh tế – Đại học Kinh tế Quốc dân
Email: hoanpm@neu.edu.vn

MỤC TIÊU

- *Trình bày khái niệm tính đa hình, và phương thức ảo.*
- *Trình bày khái niệm lớp cơ sở ảo, hàm tạo và hàm hủy của lớp cơ sở ảo.*
- *Trình bày khái niệm khuôn hình, xây dựng khuôn hình hàm và lớp.*

NỘI DUNG CHƯƠNG 4

○ Phương thức ảo và tương ứng bội

- Định nghĩa phương thức ảo.
- Quy tắc gọi phương thức ảo.
- Tương ứng bội
- Liên kết động

○ Lớp cơ sở ảo

- Lớp cơ sở ảo và phương thức ảo
- Hàm tạo, hàm hủy và lớp cơ sở ảo

○ Khuôn hình

- Khuôn hình hàm
- Khuôn hình lớp

PHƯƠNG THỨC ẢO VÀ TÍNH ĐA HÌNH

KHÁI NIỆM ĐA HÌNH

- Tổng quát, tính đa hình có nghĩa là nhiều hình thức. Thông thường, tính đa hình xảy ra khi có sự phân cấp lớp dựa trên tính kế thừa.
- Lập trình hướng đối tượng, tính đa hình có nghĩa là một lời gọi tới một hàm thành viên sẽ gọi tới một hàm khác để thực thi tùy thuộc vào kiểu của đối tượng gọi hàm.

PHƯƠNG THỨC ẢO VÀ TÍNH ĐA HÌNH

VÍ DỤ

```
1. #include <iostream>
2. using namespace std;
3. class Father
4. {
5.     ...
6.     public:
7.         int area()
8.         {
9.             cout << "This is Father!" << endl;
10.        }
11. };
```

PHƯƠNG THỨC ẢO VÀ TÍNH ĐA HÌNH

VÍ DỤ

```
12. class Son: public Father
13. {
14.     ...
15.     public:
16.         int area ()
17.         {
18.             cout << "This is Son!" <<endl;
19.         }
20. };
```

PHƯƠNG THỨC ẢO VÀ TÍNH ĐA HÌNH

VÍ DỤ

```
21. class Daughter: public Father
22. {
23.     ...
24.     public:
25.         int area ()
26.         {
27.             cout << "This is Daughter!" <<endl;
28.         }
29. };
```

PHƯƠNG THỨC ẢO VÀ TÍNH ĐA HÌNH

VÍ DỤ

```
30. int main( )
31. {
32.     Father *fa;
33.     Son son;
34.     Daughter dau;
35.     fa = &son;    // lưu trữ địa chỉ của son
36.     fa->area();   // gọi hàm area() của son
37.     fa = &dau;    // lưu trữ địa chỉ của dau
38.     fa->area();   // gọi hàm area của daughter
39.     return 0;
40. }
```


PHƯƠNG THỨC ẢO VÀ TÍNH ĐA HÌNH

VÍ DỤ

- Kết quả của chương trình:
This is Father!
This is Father!

PHƯƠNG THỨCẢO VÀ TÍNH ĐA HÌNH

ĐẶT VẤN ĐỀ

- Lý do đầu ra không đúng là do lời gọi đến hàm `area()` đã được thiết lập trước bởi trình biên dịch như là phiên bản đã định nghĩa trong lớp cơ sở. Đây được gọi là **“phân giải tĩnh”** của lời gọi hàm hay **“liên kết tĩnh”** – lời gọi hàm đã được cố định trước khi chương trình được thực hiện.
- Điều này đôi khi cũng được gọi là **“ràng buộc sớm”** bởi vì hàm `area()` được thiết lập khi biên dịch chương trình.

PHƯƠNG THỨC ẢO VÀ TÍNH ĐA HÌNH

PHƯƠNG THỨC ẢO

- Một hàm ảo là một hàm của lớp cơ sở được khai báo sử dụng từ khóa **virtual**.
- Định nghĩa trong một lớp cơ sở một hàm ảo, với các phiên bản khác nhau trong một lớp dẫn xuất, báo hiệu cho trình biên dịch rằng chúng ta không muốn “**liên kết tĩnh**” cho hàm này.
- Định nghĩa này cho phép chúng ta gọi đến hàm của đối tượng mà con trỏ trỏ tới không phụ thuộc vào kiểu của con trỏ. Điều này được gọi là “**liên kết động**” hay “**ràng buộc sau**”.

PHƯƠNG THỨC ẢO VÀ TÍNH ĐA HÌNH

PHƯƠNG THỨC ẢO - VÍ DỤ

```
1. #include <iostream>
2. using namespace std;
3. class Father
4. {
5.     ...
6.     public:
7.         virtual int area()
8.         {
9.             cout << "This is Father!" << endl;
10.        }
11. };
```

PHƯƠNG THỨCẢO VÀ TÍNH ĐA HÌNH

VÍ DỤ

- Kết quả của chương trình sau khi sửa đổi sẽ là:

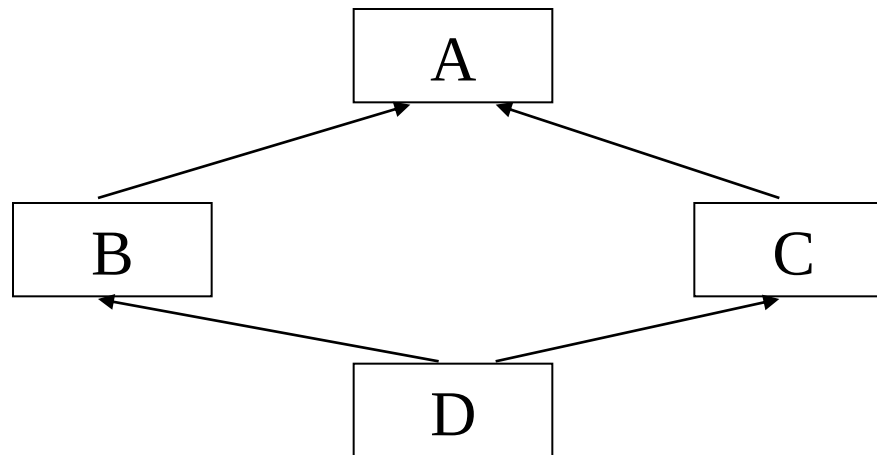
This is Son!

This is Daughter!

LỚP CƠ SỞ ẢO

ĐẶT VẤN ĐỀ

○ Xét sơ đồ kế thừa:



- Trong đó: lớp A được kế thừa bởi hai lớp B và C. Lớp D kế thừa trực tiếp cả hai lớp B và C.

LỚP CƠ SỞ ẢO

KHÁI NIỆM

- Như vậy lớp A được lớp D kế thừa *hai lần*:
 - Lần thứ nhất nó được kế thừa thông qua lớp B.
 - Lần thứ hai được kế thừa thông qua lớp C.
 - ⇒ Có 2 bản sao của lớp A trong lớp D.
- Để giải quyết tình không rõ ràng này, C++ có một cơ chế mà nhờ đó chỉ có một bản sao của lớp A ở trong lớp D: đó là sử dụng lớp ***cơ sở ảo***.

LỚP CƠ SỞ ẢO

ĐỊNH NGHĨA

- `class A`
- `{ ...};`
- `class B : virtual public A`
- `{ ...};`
- `class C : virtual public A`
- `{ ...};`
- `class D : public B, public C`
- `{ ...};`

LỚP CƠ SỞ ẢO

ĐỊNH NGHĨA

- Việc chỉ định A là ảo trong các lớp B và C nghĩa là A sẽ chỉ xuất hiện một lần trong lớp D. Khai báo này không ảnh hưởng đến các lớp B và C.
- Từ khóa *virtual* có thể đặt trước hoặc sau từ khóa *public*, *private*, *protected*.

LỚP CƠ SỞ ẢO

VÍ DỤ

```
#include <iostream>
#include <cstdlib>
#include <conio.h>
using namespace std;
class A
{
    float x,y;
public:
    void set(float x1, float y1)
        { x = x1; y = y1;}
    float getx()
        { return x; }
    float gety()
        { return y; }
};
```

LỚP CƠ SỞ ẢO

VÍ DỤ

```
class B : virtual public A  
{  
};
```

```
class C : virtual public A  
{  
};
```

```
class D : public B, public C  
{  
};
```

LỚP CƠ SỞ ẢO

VÍ DỤ

```
main()
{
    D d;
    cout<<endl<<"d.B::set(2,3)\n"; d.B::set(2,3);
    cout<<endl<<"d.C::getx() = "; cout<<d.C::getx();
    cout<<endl<<"d.B::getx() = "; cout<<d.B::getx();
    cout<<endl<<"d.C::gety() = "; cout<<d.C::gety();
    cout<<endl<<"d.B::gety() = "; cout<<d.B::gety();
}
```

LỚP CƠ SỞ ẢO

VÍ DỤ

```
cout<<endl<<"d.C::set(2,3)"; d.C::set(2,3);  
cout<<endl<<"d.C::getx() = "; cout<<d.C::getx();  
cout<<endl<<"d.B::getx() = "; cout<<d.B::getx();  
cout<<endl<<"d.C::gety() = "; cout<<d.C::gety();  
cout<<endl<<"d.B::gety() = "; cout<<d.B::gety();  
getch();  
}
```

LỚP CƠ SỞ ẢO

HÀM TẠO, HÀM HỦY

- Định nghĩa hàm tạo:
 - Hàm tạo lớp cơ sở.
 - Hàm tạo của lớp dẫn xuất.
 - Thông tin được chuyển từ hàm tạo của lớp dẫn xuất sang hàm tạo của lớp cơ sở.
- Thứ tự gọi hàm tạo: Hàm tạo của một lớp ảo luôn luôn được gọi trước các hàm tạo khác.

BÀI TẬP CHƯƠNG 4

- Nhân viên trong một cơ quan được lĩnh lương theo các dạng khác nhau. Dạng người lao động hưởng lương từ ngân sách Nhà nước gọi là cán bộ, công chức (dạng biên chế). Dạng người lao động lĩnh lương từ ngân sách của cơ quan gọi là người làm hợp đồng. Như vậy hệ thống có hai đối tượng: biên chế và hợp đồng.
- Hai loại đối tượng này có đặc tính chung là viên chức làm việc cho cơ quan. Từ đây có thể tạo nên lớp cơ sở để quản lý một viên chức (lớp **Nguoi**) bao gồm mã số, họ tên, lương.
- Hai lớp kế thừa từ lớp cơ sở trên:
 - Lớp Bienche gồm các thuộc tính: hệ số lương, tiền phụ cấp chức vụ.
 - Lớp Hopdong gồm các thuộc tính: tiền công lao động, số ngày làm việc trong tháng, hệ số vượt giờ.

KHUÔN HÌNH

KHÁI NIỆM

- Khuôn hình là nền tảng của lập trình tổng quát, trong đó việc viết mã là độc lập với kiểu riêng biệt nào đó.
- Một khuôn hình là một kế hoạch hoặc công thức để tạo một hàm hoặc lớp tổng quát. Các mẫu thư viện như vòng lặp và thuật toán là ví dụ của lập trình tổng quát đã được phát triển sử dụng khái niệm khuôn mẫu.

KHUÔN HÌNH

KHUÔN HÌNH HÀM

- Dạng tổng quát của một định nghĩa khuôn mẫu hàm có dạng:

```
template <class type> ret-type func-  
name(parameter list)  
{ // body of function }
```

- Ở đây:
 - type là một tên kiểu cho một kiểu dữ liệu sử dụng bởi hàm. Tên này có thể được sử dụng trong định nghĩa hàm.

KHUÔN HÌNH

KHUÔN HÌNH HÀM – VÍ DỤ

```
1.  #include <iostream>
2.  #include <string>
3.  using namespace std;
4.  template <typename T>
5.  inline T const& Max (T const& a, T const& b)
6.  { return a < b ? b:a; }
7.  int main ()
8.  {
9.      int i = 39; int j = 20;
10.     cout << "Max(i, j): " << Max(i, j) << endl;
11.     double f1 = 13.5; double f2 = 20.7;
12.     cout << "Max(f1, f2): " << Max(f1, f2) << endl;
13.     string s1 = "Hello"; string s2 = "World";
14.     cout << "Max(s1, s2): " << Max(s1, s2) << endl;
15.     return 0;
16. }
```

KHUÔN HÌNH

KHUÔN HÌNH HÀM – VÍ DỤ

- Biên dịch và chạy chương trình, kết quả là:

`Max (i, j) : 39`

`Max (f1, f2) : 20.7`

`Max (s1, s2) : World`

KHUÔN HÌNH

KHUÔN HÌNH LỚP

- Dạng tổng quát của một khai báo khuôn mẫu lớp là:

```
template <class type> class class-name
{
    . . .
}
```

- Ở đây:
 - **Type** là tên kiểu, nó sẽ được chỉ ra khi một lớp được khởi tạo.
 - Có thể định nghĩa nhiều hơn một kiểu dữ liệu tổng quát bằng việc sử dụng danh sách dấu phẩy phân cách.

KHUÔN HÌNH

KHUÔN HÌNH LỚP – VÍ DỤ

```
1. #include <iostream>
2. #include <vector>
3. #include <cstdlib>
4. #include <string>
5. #include <stdexcept>
6.
7. using namespace std;
8.
9. template <class T>
10. class Stack {
11.     private:
12.         vector<T> elems;      // elements
13.
14.     public:
15.         void push(T const&);  // push element
16.         void pop();           // pop element
17.         T top() const;        // return top element
18.         bool empty() const{   // return true if empty.
19.             return elems.empty();
20.         }
21. };
```

KHUÔN HÌNH

KHUÔN HÌNH LỚP – VÍ DỤ

```
23. template <class T>
24. void Stack<T>::push (T const& elem) {
25.     // append copy of passed element
26.     elems.push_back(elem);
27. }
28.
29. template <class T>
30. void Stack<T>::pop () {
31.     if (elems.empty()) {
32.         throw out_of_range("Stack<>::pop(): empty stack");
33.     }
34.
35.     // remove last element
36.     elems.pop_back();
37. }
38.
39. template <class T>
40. T Stack<T>::top () const {
41.     if (elems.empty()) {
42.         throw out_of_range("Stack<>::top(): empty stack");
43.     }
44.
45.     // return copy of last element
46.     return elems.back();
47. }
```

KHUÔN HÌNH

KHUÔN HÌNH LỚP – VÍ DỤ

```
1.  int main() {
2.      try {
3.          Stack<int>          intStack;    // stack of ints
4.          Stack<string> stringStack;      // stack of strings
5.
6.          // manipulate int stack
7.          intStack.push(7);
8.          cout << intStack.top() <<endl;
9.
10.         // manipulate string stack
11.         stringStack.push("hello");
12.         cout << stringStack.top() << std::endl;
13.         stringStack.pop();
14.         stringStack.pop();
15.     }catch (exception const& ex) {
16.         cerr << "Exception: " << ex.what() <<endl;
17.         return -1;
18.     }
19. }
```

KHUÔN HÌNH

KHUÔN HÌNH LỚP – VÍ DỤ

- Biên dịch và thực hiện chương trình, kết quả là:

```
7
```

```
hello
```

```
Exception: Stack<>::pop(): empty stack
```


TÓM TẮT CHƯƠNG 4

- *Khái niệm tính đa hình, và phương thức ảo.*
- *Khái niệm lớp cơ sở ảo, hàm tạo và hàm hủy của lớp cơ sở ảo.*
- *Khái niệm khuôn hình, xây dựng khuôn hình hàm và lớp.*